Documentation of Rubik's Cube

by Duc Le, Edward Zhu, Kaan Eroltu, Matthew Gordon

1) Introduction (what is your FSM supposed to do)

Our FSM helps the user solve a Rubik's Cube after the bottom 2 layers have already been solved. The user provides 4 inputs based on the current state of the cube. Then the FSM displays instructions on how to solve the cube, which includes which color face the user should have in front, and what moves should be made, which are displayed with LEDs. There are 4 stages (Create the Yellow Cross, Align Yellow Edges, Position Yellow Corners, Solve the Cube) that the user progresses through. At the start of each stage, the user presses 4 buttons for the input. Each of the team members handled a different stage.

Stage 1 Create the Yellow Cross:
The FSM guides the user through appropriate algorithms to form the yellow cross. The user will input the information about the edges adjacent to the yellow center. If the edge is yellow, then it is 1. If not, then it is 0. Given the inputs, the FSM gives the algorithm to complete and the color of the side we are going to face. There are 4 cases - the dot, the hook, the line, and the cross. The hook requires A1 (F U R U' R' F'), and the line requires A2 (F R U R' U' F'). The dot requires the line algorithm, then 2 Us, and finally the hook algorithm (A2 U U A1). If we already have a cross, the FSM will light a green LED, indicating that this stage is finished. If there are cases that cannot be solved (when we have an odd number of edges that are yellow), the FSM will light a red LED, indicating that we have an error, and we should disassemble the cube and start from the beginning. The side that the user will face will be determined according to how the cases are oriented, and the LED with the color of the side we want to face will light up. The FSM will progress to the next state when we complete the algorithms, form the yellow cross, and then press the button.

Stage 2 Align Yellow Edges:
Once the cross is made, the FSM guides the user in performing an algorithm to match the yellow edges with the corresponding center colors on the side faces. If there is a match between the edge color and the center color, the user inputs 1, otherwise, they input 0. Prior to this stage, the user is asked to perform U moves until they get at least 2 matches, which is always possible. If the user inputs an invalid state (0, 1, or 3 matches, which are impossible states), an LED for "Error" will light up. If there are 4 matches, an LED for "Complete" will light up.

Based on the input, the FSM outputs instructions to position the cube such that they are facing a certain color, and to perform either the Sune algorithm (R U R' U R U2 R') or a modified algorithm (Sune, then U', then Sune again). Each of these algorithms will be displayed with an

LED. After they perform either of these 2 algorithms, then there should be 4 matches, and the stage is complete.

Stage 3 Position Yellow Corners:
The FSM identifies whether any corners are in their correct positions (though not necessarily correctly oriented). If one or more corners are correctly positioned, it instructs the user to hold the cube in a specific orientation and apply algorithms.

There are two sub-stages for this stage, since there must be 2 algorithms done back to back to ensure the completion of this stage. The first substage identifies whether any corner is in correct position or not, and stores that status into 1 flip-flop. If all corners are already corrected, the output would be "do nothing". Else, the machine instructs the user to face the corresponding face and perform (R U L U' R' U L' U') in case of one correct corner OR in case of no correct corners. Any other inputs will result in an error.

There must always be at least one correct corner for the second substage. If every corner is solved, the machine outputs "Do nothing". If the cube was unsolved in the previous substage AND there is one solved corner in this substage, the machine tells the user to face the corresponding face and do (R U L U' R' U L' U') again. The only special case is when no corners were solved in the last substage AND the correct corner in this substage is in the middle of orange and green centers. For this case, the machine outputs "Green" and tells the user to do a special combination (L U' R U L' U' R' U). That is why the memory bit from the first substage was important.

After these two substages, the corners should all be aligned, and the stage is complete.

Stage 4 Solve the Cube:
In the final stage, the FSM helps the user orient the corners of the yellow face (the top face) so that all corner pieces have the yellow side facing up. To begin, the user provides 4 inputs—one for each corner of the yellow face. If a corner is already yellow-side up, the input is 1; otherwise, it is 0.

The FSM processes this input and outputs a sequence of 8 LEDs, each corresponding to a specific step in an algorithmic sequence. The format of these instructions follows a repeating pattern of Lefty (L' U' L U) and U turns. A lit LED indicates that the corresponding algorithm should be applied.

If a "Lefty" LED is lit, the user should repeatedly apply the Lefty algorithm (L' U' L U) until the corner on the right of the front-facing side is yellow-side up, and this takes 2 or 4 repetitions. Between each application of Lefty algorithms, the user performs a U move to shift the next

unsolved corner into position. This loop continues until all four corners are correctly oriented. Once all corners are yellow-side up, the cube is fully solved, and the FSM signals completion.

2) Methods (how does it do it? What design choices did you make?)

For inputs, we have a total of 8 inputs. 4 of the inputs are related to the edges of the yellow side, and the other 4 inputs are related to the corners of the yellow side. The 4 inputs of the edges are used in Stage 1 and Stage 2, with Stage 1 representing if the edges are yellow and Stage 2 representing if the top edge of the surrounding faces match the center color of the face. The other 4 inputs of the corners are used in Stage 3 and Stage 4, with Stage 3 representing if the entire corner block contains 3 colors that are the 3 colors of the center of the sides containing that corner block, although not necessarily directly matching with the sides. Stage 4 uses the inputs of the corners to represent if the corner is completely solved, with the colors directly matching the sides.

We use 2 main types of output: Face orientation and algorithm instruction. For face orientation output, a 2:4 decoder is used to select which side (Red, Blue, Green, or Orange) the user should face. The Yellow face is always on top, and the White is always on the bottom, so they are not included. Each stage uses this orientation cue.

For the algorithms, there are 4 LEDs used for Stages 1 and 2 (triggered by a 2:4 decoder), 5 LEDs for Stage 3, and 8 LEDs for Stage 4. These LEDs are labelled with a piece of paper which shows what the moves are.

For the stage progression, we used a 3-bit binary counter to represent each stage, from 001 to 101 (note that Stage 3 uses both 011 and 100), and we reset the counter at 110. The outputs from the counter is passed to a 3:8 decoder, with each output passed to the clock input of the D flip-flop representing the output of the corresponding stage, so that we can update the output when we reach the stage (clock rises from 0 to 1). The output of 110 from the decoder is used to reset the counter. This enables us to progress between different states.

Each of the 4 stages uses combinational logic to determine the output. Each stage's combinational logic was built on separate breadboards, then at the end combined together.

3) Self-Assessment (who did what?)

Duc handled the combinational logic and built the circuit for stage progression and the decoder for the face colors. Matthew handled the combinational logic and built the circuit for Stage 1. Edward and Kaan did the same for Stage 2 and Duc did the same for Stage 3. Kaan and Edward handled the combinational logic for Stage 4, and Duc and Edward built the circuit for Stage 4.

4) A screenshot of your simulation and a photo of your hardware circuits